```
% git commit -m "add the file hello"

[master bec274f52] add the file hello
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt
```

Update the *hello.txt* file by adding a second line to it:

```
% cat >> hello.txt

This is the second line

% cat hello.txt  #

This is the first line
This is the second line
```

Add and commit the same file again:

```
% git add hello.txt
% git commit -m "added second line in the file hello"

[master 3af6fea8d] added second line in the file hello
 1 file changed, 1 insertion(+)
```

Check the log and search for two git commits.

```
% git log

commit 3af6fea8dbb44973ddeed856036d0792d15a4ad7 (HEAD ->
master)
Author: xxxxxxx <xxxxxxx@xxxxxxx>
Date:   Sun Jun 13 20:29:00 2021 +0530
    added second line in the file hello

commit bec274f5264140c9ff1f9b8b6f956c69295a29d4
Author: xxxxxxx <xxxxxxx@xxxxxxx>
Date:   Sun Jun 13 20:27:39 2021 +0530
    add the file hello
```

Let's now reset the head version of the file to the previous commit:

```
% git reset --hard bec274f5264140c9ff1f9b8b6f956c69295a29d4

HEAD is now at bec274f52 add the file hello
```

Check the git log again; you will find the last created commit has vanished.

```
% git log

commit bec274f5264140c9ff1f9b8b6f956c69295a29d4 (HEAD ->
master)
Author: xxxxxxx <xxxxxxx@xxxxxxx>
Date:   Sun Jun 13 20:27:39 2021 +0530
    add the file hello
```

Next, we will try to find the dangling commit.

```
% git fsck --lost-found

Checking object directories: 100% (256/256), done.
Checking objects: 100% (66417/66417), done.
dangling commit 3af6fea8dbb44973ddeed856036d0792d15a4ad7
```

Match the details and check if it was the same commit:

```
% git show 3af6fea8dbb44973ddeed856036d0792d15a4ad7

commit 3af6fea8dbb44973ddeed856036d0792d15a4ad7
Author: xxxxxxx <xxxxxxx@xxxxxxx>
Date:   Sun Jun 13 20:29:00 2021 +0530
    added second line in the file hello
diff --git a/hello.txt b/hello.txt
index d3e2104b9..81bc58de5 100644
--- a/hello.txt
+++ b/hello.txt
    @@ -1 +1,2 @@
 This is the first line
+This is the second line
```

When *prune* is run now, it may have no effect as git may be maintaining the reference and not be fully detached. Run the *reflog* to expire all the entries that are older than now. It is also advised to run *git gc* rather than *git prune*.

```
% git reflog expire --expire=now --expire-unreachable=now
--all
```

Remember to dry run the *prune* command before running it. This will let you check the changes that will take place by running *prune:*

```
% git prune --dry-run --verbose --progress --expire=now

142cc07007d0107e8ff91b28e2a7b078a6f56d1c tree
3af6fea8dbb44973ddeed856036d0792d15a4ad7 commit
81bc58de5525b0c16f8cb74b89799017b6986981 blob
```

If you find the changes are as per your expectations, you can run the actual *prune* command:

```
% git prune --verbose --progress --expire=now
```